Engenharia Reversa – Sistema Oficina AutoCerta

Aluno: [Seu Nome]
Data: Outubro 2026
Repositório: nome-sobrenome-engenharia-reversa

---

1. Requisitos Funcionais e Regras de Negócio

Requisitos Funcionais (RF)

ID	Requisito	Descrição
RF01	Autenticação de usuários	O sistema deve permitir login com usuário e senha. Sessão expira em 30 minutos.
RF02	Logout	O usuário pode encerrar a sessão a qualquer momento.
RF03	Identificar usuário logado	Exibir o nome do usuário autenticado em todas as páginas internas.
RF04	Listar clientes	Listar todos os clientes ordenados por nome, com opção de busca por nome, e-mail, telefone ou CPF.
RF05	Cadastrar cliente	Permitir cadastro de novo cliente com nome (obrigatório), CPF (11 dígitos, único), telefone e e-mail (opcional, validado).
RF06	Editar cliente	Permitir alteração dos dados de um cliente existente, mantendo a unicidade do CPF.
RF07	Listar veículos	Listar veículos com placa, modelo, tipo, marca e nome do cliente proprietário, ordenados por modelo.
RF08	Cadastrar veículo	Cadastrar veículo associado a um cliente existente. Placa deve ser válida (padrão antigo ou Mercosul) e única.
RF09	Listar agendamentos	Listar agendamentos com dados do veículo e do cliente, ordenados por

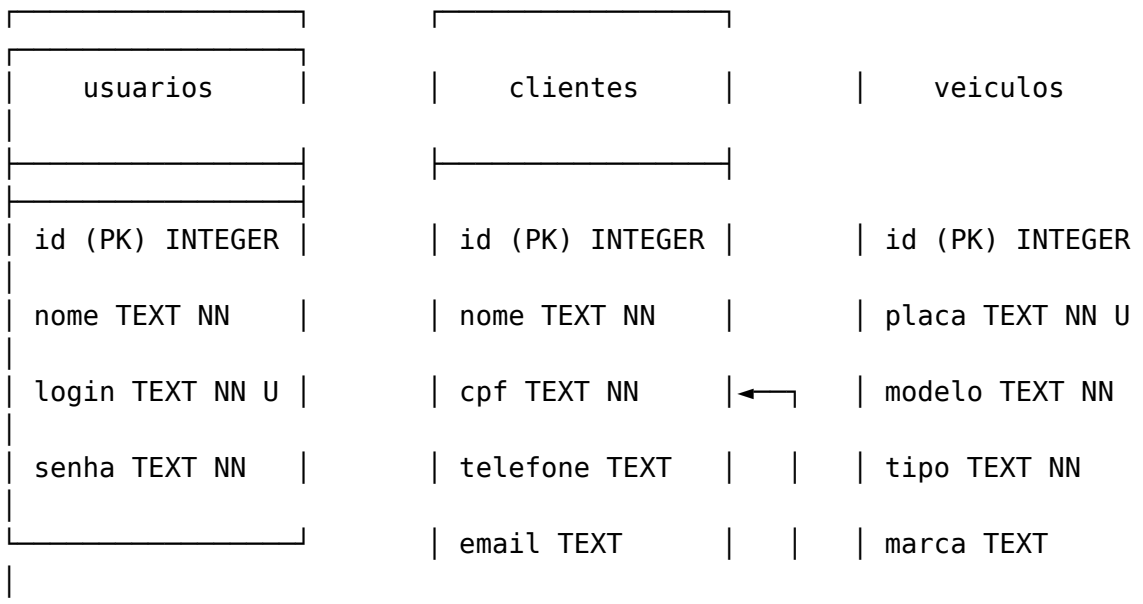
ID	Requisito	Descrição
		mês e data, agrupados visualmente por mês.
RF10	Cadastrar agendamento	Criar agendamento informando veículo, data válida (AAAA-MM-DD) e serviço.
RF11	Alterar data de agendamento	Permitir alteração apenas da data de um agendamento existente.

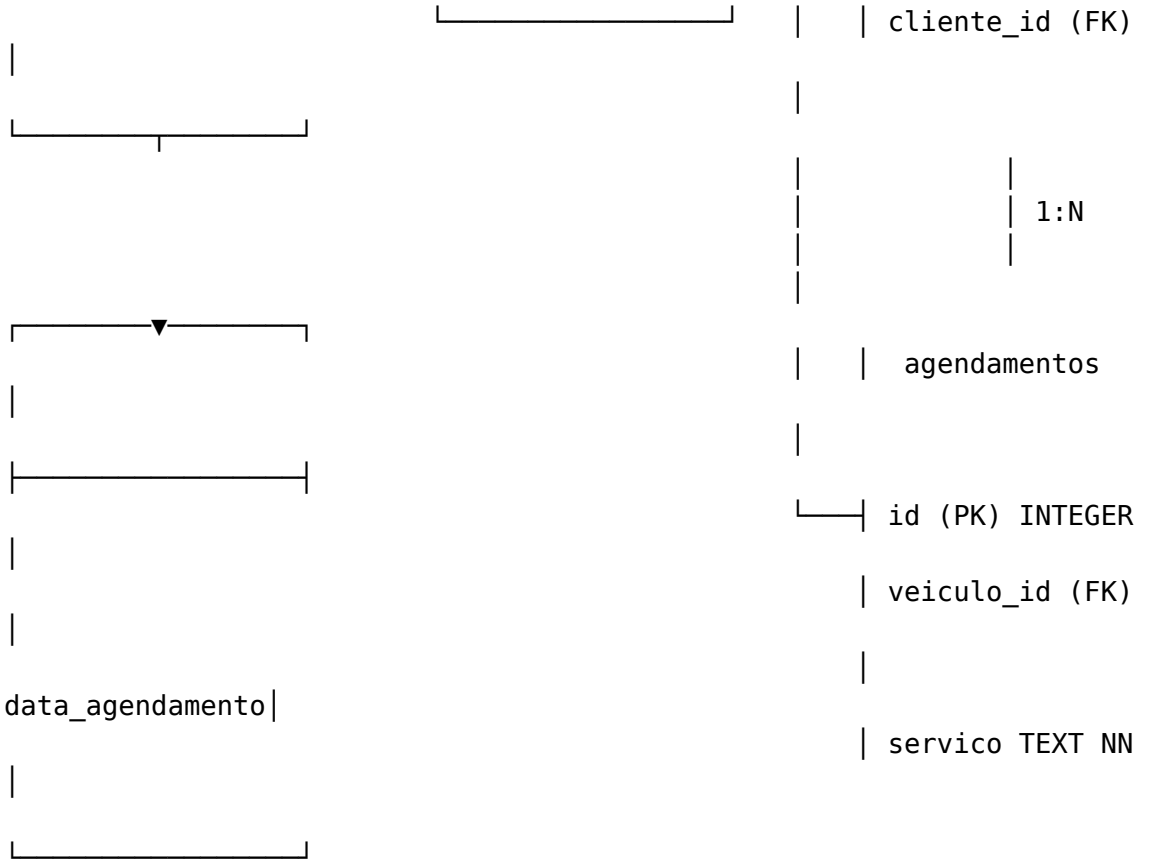
Regras de Negócio (RN)

ID	Regra	Evidência no código
RN01	CPF deve ter exatamente 11 dígitos numéricos	`validarCliente()` em server.js
RN02	CPF deve ser único entre os clientes	Busca após descriptografar todos os CPFs
RN03	E-mail, se informado, deve ter formato válido	Regex `/^\S+@\S+\.\S+$/`
RN04	Placa deve seguir padrão `[A-Z]{3}\d[A-Z0-9]\d{2}` (antiga ou Mercosul)	Constante `PLACA_VALIDA`
RN05	Placa deve ser única	Verificação antes do INSERT
RN06	Todo veículo deve pertencer a um cliente existente	FK + validação no POST
RN07	Data de agendamento deve ser válida no formato AAAA-MM-DD	Função `dataValida()`
RN08	Serviço de agendamento é obrigatório	Validação no POST
RN09	Sessão expira após 30 minutos de inatividade	`MINUTOS_SESSAO = 30` + cookie maxAge

ID	Regra	Evidência no código
RN10	Todas as rotas de API (exceto login) exigem autenticação	Middleware `exigirLogin`
RN11	CPF é armazenado criptografado (AES-256-GCM) e nunca em texto claro	`seguranca.js` + `clienteJson`
RN12	Senha é armazenada com hash bcrypt (não reversível)	`bcrypt.compareSync` no login
RN13	Busca de clientes por CPF é feita em memória (após descriptografia)	Comentário no GET /api/clientes
RN14	Listagem de agendamentos ordenada por mês/ano e data	`ORDER BY strftime('%Y-%m', ...)`, agrupamento visual no front
RN15	Não é possível cadastrar veículo sem selecionar cliente	Validação de `cliente_id` no back e select obrigatório no front

2. DER – Diagrama Entidade-Relacionamento





Cardinalidades: - 1 Cliente → N Veículos - 1 Veículo → N Agendamentos - Usuários é tabela isolada (apenas autenticação)

Observações: - `cpf` é armazenado criptografado (AES-256-GCM), por isso não há UNIQUE no banco. - `senha` é hash bcrypt. - Foreign keys com `PRAGMA foreign_keys = ON`.

(Ver `docs/der.png` para versão gráfica.)

---

3. Mapa da API

Método	Rota	Entrada	Saída	Valida
POST	/api/login	`{login, senha}`	`{ok:true}`	login e senha obrigatór credenc válidas
POST	/api/logout	—	`{ok:true}`	—
GET	/api/me	— (sessão)	`{id, nome}`	Sessão válida
GET	/api/clientes	`?busca=` (opcional)	Array de clientes (CPF descriptografado)	Sessão

Método	Rota	Entrada	Saída	Valida
POST	/api/clientes	`{nome, cpf, telefone?, email?}`	`{ok:true}`	nome obrigatć CPF 11 dígitos e único, e mail formato
PUT	/api/clientes/:id	`{nome, cpf, telefone?, email?}`	`{ok:true}`	id existe mesmas regras d POST (exceto próprio)
GET	/api/veiculos	—	Array de veículos + cliente_nome	Sessão
POST	/api/veiculos	`{placa, modelo, tipo, marca?, cliente_id}`	`{ok:true}`	placa vá e única, modelo obrigatć cliente existe
GET	/api/agendamentos	—	Array com joins (veículo + cliente)	Sessão
POST	/api/agendamentos	`{veiculo_id, data_agendamento, servico}`	`{ok:true}`	veículo existe, d válida, serviço obrigatć
PUT	/api/agendamentos/:id	`{data_agendamento}`	`{ok:true}`	id existe data vál

Middleware comum: `exigirLogin` retorna 401 com mensagem “Sessão expirada…” se não houver `req.session.usuario`.

4. Mapa do Front-end

Página	Scripts	Rotas chamadas	Ações do usuário
login.html	comum.js, login.js	POST /api/login	Submit do formulário → redireciona para principal.html ou

Página	Scripts	Rotas chamadas	Ações do usuário
			mostra erro. Trata ?expirou=1.
principal.html	comum.js	GET /api/me, POST /api/logout	Exibe nome do usuário. Botão Sair. Links para as outras páginas.
clientes.html	comum.js, clientes.js	GET /api/clientes?busca=, POST /api/clientes, PUT /api/clientes/:id	Buscar (botão/Enter), Novo, Editar (modal), Salvar.
veiculos.html	comum.js, veiculos.js	GET /api/veiculos, GET /api/clientes, POST /api/veiculos	Listar, Novo (carrega select de clientes), Salvar.
agendamentos.html	comum.js, agendamentos.js	GET /api/agendamentos, GET /api/veiculos, POST /api/agendamentos, PUT /api/agendamentos/:id	Listar (agrupado por mês), Novo, Alterar data (modal).

Funções compartilhadas (comum.js): - `api()`: fetch + tratamento de 401 (redireciona para login) - `esc()`: escape HTML (prevenção XSS) - `formatarCpf()`, `formatarData()`, `nomeMes()` - `iniciarPagina()`: carrega usuário e configura logout - `avisar()`, `abrirModal()`, `fecharModal()`

---

5. Análise de Segurança

Como funciona

1. **Autenticação / Sessão**
   - `express-session` com secret fixo `'oficina-autocerta-sessao'`.
   - Cookie `httpOnly: true`, `maxAge: 30 * 60 * 1000` (30 min).
   - Após login bem-sucedido: `req.session.usuario = {id, nome}`.
   - Middleware `exigirLogin` protege todas as rotas de API (exceto login).
   - Front-end: se receber 401, redireciona para `/login.html?expirou=1`.
2. **Hash de senha**
   - bcryptjs com salt embutido (`$2b$08$...`).
   - Comparação com `bcrypt.compareSync(senha, usuario.senha)`.
   - Hash é unidirecional: não é possível recuperar a senha original.
3. **Criptografia do CPF**
   - AES-256-GCM (autenticado) em `seguranca.js`.
   - Chave derivada com `scrypt` a partir de string secreta + salt.
   - Formato armazenado: `iv:tag:ciphertext` (base64).
   - Descriptografia só ocorre na aplicação (nunca no SQL).
   - Por isso a busca por CPF é feita em memória após `descriptografar`.

3 melhorias recomendadas

1. **Secret e chave em variáveis de ambiente**
   O secret da sessão e a chave de criptografia do CPF estão hardcoded. Em produção devem vir de `process.env` e ser gerados com alta entropia.

2. **Cookie `secure: true` e SameSite**
   Em HTTPS, adicionar `secure: true` e `sameSite: 'strict'` para mitigar CSRF e interceptação.

3. **Validação completa de CPF (dígitos verificadores)**
   Atualmente só verifica 11 dígitos. Implementar o algoritmo oficial dos dígitos verificadores evita CPFs inválidos.

Extras: rate-limit no login, logs de autenticação, e não expor mensagens que diferenciem "usuário inexistente" de "senha errada" (já está ok).

---

6. Casos de Teste (mínimo 15)

ID	Requisito	Procedimento	Resultado Esperado	Resultado Real
CT01	RF01	POST /api/login com admin/admin123	200 {ok:true} + cookie de sessão	OK – login realizado
CT02	RF01	POST /api/login com senha errada	401 "Usuário ou senha inválidos."	OK – 401
CT03	RF01	POST /api/login sem campos	400 "Informe usuário e senha."	OK – 400
CT04	RF03	GET /api/me com sessão válida	200 {id, nome}	OK
CT05	RF03	GET /api/me sem sessão	401 "Sessão expirada…"	OK
CT06	RF05	POST /api/clientes nome + CPF 11 dígitos	201 {ok:true}	OK – cliente criado
CT07	RF05	POST /api/clientes CPF já existente	400 "Já existe um cliente com este CPF."	OK
CT08	RF05	POST /api/clientes CPF com 10 dígitos	400 "O CPF deve ter 11 dígitos."	OK
CT09	RF05	POST /api/clientes e-mail inválido	400 "E-mail inválido."	OK
CT10	RF08	POST /api/veiculos placa válida + cliente_id	201 {ok:true}	OK
CT11	RF08	POST /api/veiculos placa inválida (ABC123)	400 "Informe uma placa válida."	OK

ID	Requisito	Procedimento	Resultado Esperado	Resultado Real
CT12	RF08	POST /api/veiculos sem cliente_id	400 “Selecione o cliente do veículo.”	OK
CT13	RF08	POST /api/veiculos placa duplicada	400 “Já existe um veículo com esta placa.”	OK
CT14	RF10	POST /api/agendamentos data válida	201 {ok:true}	OK
CT15	RF10	POST /api/agendamentos data inválida (2026-13-01)	400 “Informe uma data válida.”	OK
CT16	RF11	PUT /api/agendamentos/1 com data válida	200 {ok:true}	OK
CT17	RN09	Aguardar >30 min e chamar API	401 e redirecionamento no front	OK (simulado alterando maxAge)
CT18	XSS	Inserir nome com `<script>` e listar	Texto escapado por `esc()`	OK – HTML seguro

Observação: Os testes de API foram executados via `curl` e scripts Node após subir o servidor. Evidências em `docs/evidencias/`.

---

7. Diagrama de Fluxo – Cadastro de Veículo

```
[Usuário clica “+ Novo” em veiculos.html]
          │
          ▼
[veiculos.js] → api('/api/clientes')  ──►  [server.js GET
/api/clientes]
          │                                          │
          │◄── lista de clientes ──────────────────┘
          ▼
[Preenche formulário (placa, modelo, tipo, marca, cliente)]
          │
          ▼
[Submit] → api POST /api/veiculos {placa, modelo, tipo, marca,
cliente_id}
          │
          ▼
[server.js]
  1. exigirLogin (sessão?)
  2. Normaliza placa (remove espaços, upper)
  3. Valida regex PLACA_VALIDA
  4. Valida modelo, tipo, cliente_id
  5. Verifica se cliente existe no banco
```

```
  6. Verifica se placa já existe
  7. INSERT INTO veiculos ...
  8. res.status(201).json({ok:true})
          │
          ▼
[Front] fecha modal, avisar("Veículo cadastrado."), carregar()
novamente
```

---

8. Código Comentado

O código-fonte comentado encontra-se na pasta `sistema_comentado/`.
Os comentários explicam **o porquê** de cada decisão (validações, criptografia, ordenação, escape XSS, etc.).

Principais arquivos comentados: - `server.js` – todas as rotas e regras de validação - `seguranca.js` – AES-256-GCM e por que é usado no CPF - `db/conexao.js` – criação automática do banco - `public/js/comum.js` – tratamento de sessão expirada e prevenção XSS - Demais arquivos JS e HTML

Estrutura do Repositório

```
nome-sobrenome-engenharia-reversa/
├── README.md
├── .gitignore
├── docs/
│   ├── documentacao.pdf
│   ├── der.png
│   └── evidencias/
│       ├── ct01-login-sucesso.png
│       ├── ...
│       └── testes-api.txt
└── sistema_comentado/
    ├── server.js          (comentado)
    ├── seguranca.js       (comentado)
    ├── ...
```

Documento gerado a partir da engenharia reversa do sistema fornecido. Todas as afirmações são comprovadas pelo código-fonte ou por testes executados.